

Neuro-Cognitive Grounding: Modeling How a Human-Like Reasoner Learns from Examples

A complete ARC-style reasoning system should not only search over symbolic programs; it should also model how an intelligent observer forms internal representations, updates beliefs from examples, and preserves useful abstractions across tasks. This places the system at the intersection of **computational neuroscience**, **cognitive modeling**, **Bayesian inference**, **information geometry**, and **symbolic program synthesis**.

Computationally modeling how the human brain observes a task and learns from examples is the domain of computational neuroscience and cognitive modeling. In this view, scientists translate biological learning principles into mathematical algorithms that simulate how neurons encode stimuli, adapt their responses, and adjust their connections. For ARC-style reasoning, this does not require fully simulating the brain; rather, it means borrowing biologically and cognitively grounded mechanisms for representation, prediction, memory, and adaptation.

1. Neural and Cognitive Representation

Before solving a task, the system must convert raw grid examples into structured internal representations. A human does not see only pixels; the visual system organizes the scene into objects, colors, shapes, spatial relations, symmetries, and possible transformations. Similarly, an ARC solver should convert examples into symbolic, vector, graph, or probabilistic representations.

A biologically inspired route would use **spiking neural networks**, where information is communicated through discrete spike events rather than continuous activations. This is closer to how neurons communicate in the brain and can support energy-efficient temporal computation. A cognitive modeling route can instead use **Gaussian processes**, graph-based representations, object-centric encoders, or probabilistic programs to represent abstract concepts such as object permanence, containment, causality, symmetry, and transformation.

In the ARC system, this becomes the representation layer:

```
Grid examples
  → object segmentation
  → object/relation graph
  → visual predicates
  → candidate rule distributions
  → reasoning trace distribution
```

This connects directly to information geometry: the solver does not merely store raw features; it stores probability distributions over possible rules, objects, relations, and transformations.

2. Synaptic Plasticity and Learning Rules

Human learning depends on changes in connection strength between neurons. A computational reasoning system can borrow this idea through adaptive weights, memory updates, and belief revision.

Two important learning principles are:

Hebbian learning: neurons that activate together strengthen their connection. In ARC terms, if a visual pattern repeatedly co-occurs with a successful operator, the association between that pattern and operator should strengthen. For example, if “same shape but different color” often predicts a recoloring operation, the system should increase the prior probability of recoloring when that visual relation appears again.

Predictive coding: the system generates a prediction, compares it with the observed output, computes an error, and updates its internal model to reduce future error. In ARC, every candidate program predicts an output grid. The mismatch between predicted and target output becomes a structured prediction error. Near-solved failures are especially valuable because they reveal which part of the reasoning trace was correct and which transformation was missing.

So ARC learning becomes:

```
predict output
compare with target
localize error
update belief over operators
store failure pattern
invent operator if error pattern repeats
```

This makes failure productive rather than wasteful.

3. Reward, Imitation, and Bayesian Belief Updating

Humans often learn by observing successful behavior, inferring hidden goals, and updating beliefs about what action caused success. A reasoning system can do something similar.

In ARC, the examples act as demonstrations. The solver observes input-output pairs and asks:

```
What hidden rule could have produced this output from this input?
```

This is a Bayesian inference problem:

$$p(\text{rule} \mid \text{examples}) \propto p(\text{examples} \mid \text{rule})p(\text{rule})$$

The prior $p(\text{rule})$ comes from memory: previously solved tasks, known operators, and recurring visual structures. The likelihood $p(\text{examples} \mid \text{rule})$ measures how well a candidate program explains the demonstrations.

Bayesian optimization then becomes the active search controller. It chooses which candidate rule or operator to test next by balancing expected success and uncertainty:

```
high expected fit + useful uncertainty → test next  
low fit + low uncertainty → ignore  
high uncertainty near repeated failure cluster → possible new operator
```

This gives the solver a cognitively plausible strategy: it does not brute-force everything equally. It explores hypotheses that are informative.

4. Category Theory and Topos-Like Visual Logic

Category theory provides the compositional backbone of the reasoning system. ARC transformations can be treated as typed morphisms between structured objects:

```
Grid → ObjectSet → RelationGraph → TransformedObjectSet → OutputGrid
```

Each operator has an input type, output type, precondition, postcondition, and composition rule. This prevents invalid reasoning chains and allows complex solutions to be built from smaller reusable transformations.

Topos theory adds a logic-geometric bridge. In ARC, visual objects can be translated into propositions:

```
red(x)  
square(x)  
inside(x, frame)  
same_shape(x, y)  
touches_border(x)  
larger_than(x, y)
```

A visual transformation can then be represented both geometrically and logically:

```
Move the red square into the empty frame.
```

becomes:

$$Red(x) \wedge Square(x) \wedge EmptyFrame(y) \Rightarrow MoveTo(x, y)$$

This lets the solver move between shapes, predicates, programs, and proofs.

5. Information Geometry as Memory Shape

Information geometry gives the system a way to preserve and compare reasoning memories. Instead of storing only a solved program, the solver stores the distributional shape of the reasoning process:

```
operator distribution
object-role distribution
relation distribution
transformation-parameter distribution
failure distribution
importance weights
verification certificate
```

This is important because two tasks may look different at the pixel level but require the same abstract rule. Information geometry compares tasks by belief behavior rather than raw appearance.

A solved task becomes a memory region. A near-solved task becomes a geometric gap. If many failures cluster in the same region of belief space, the system can infer that it lacks a reusable operator.

6. Practical Tools and Implementation

For biologically realistic modeling, tools such as Brian/Brian2-style simulators can be used to build spiking neural networks and synaptic plasticity experiments. Nengo can be used to construct larger-scale cognitive models that connect neural representations to symbolic or motor-like computations.

For a practical ARC solver, however, the first implementation should use a cognitive abstraction rather than a full neuron-level simulation:

```
object-centric perception
probabilistic belief distributions
Bayesian program search
typed categorical DSL
topos-style predicate logic
information-geometric memory
certificate-based verification
```

The biologically realistic version can be added later as an experimental module that tests whether spiking or neural-symbolic dynamics can reproduce the same reasoning traces.

Integrated Thesis

The resulting system is a neuro-cognitive, geometric, and symbolic reasoning architecture. Computational neuroscience motivates how examples become internal representations and adaptive memories. Cognitive modeling explains how the system forms hypotheses, predicts outputs, and revises beliefs. Bayesian inference and Bayesian optimization guide rule search under uncertainty. Category theory provides

compositional structure. Topos-style logic translates visual objects into propositions. Information geometry preserves the shape of solved and failed reasoning traces.

In this framework, an ARC solver does not merely search for programs. It observes examples, forms structured beliefs, predicts outputs, localizes errors, updates memory, invents missing operators, and verifies them through symbolic execution.